

Testing HTTP-based APIs using RestAssured.Net

An open source workshop by ...

What are we going to do?

- _HTTP-based APIs

- _RestAssured.Net

- _Hands-on exercises

Preparation

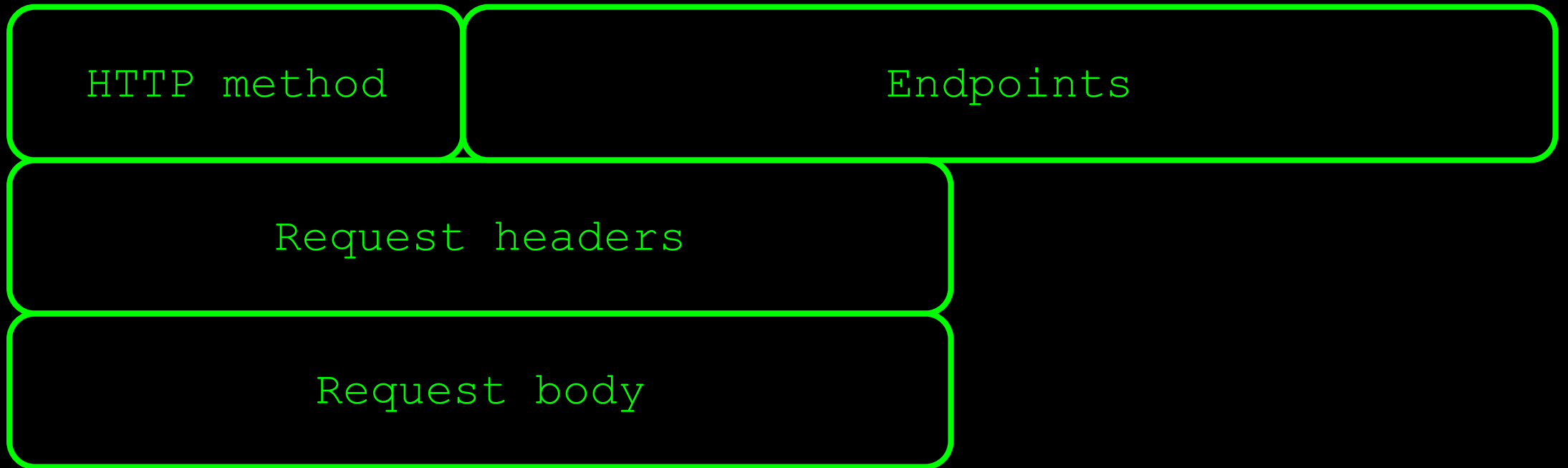
_Install a recent .NET SDK (.NET 8 or newer)

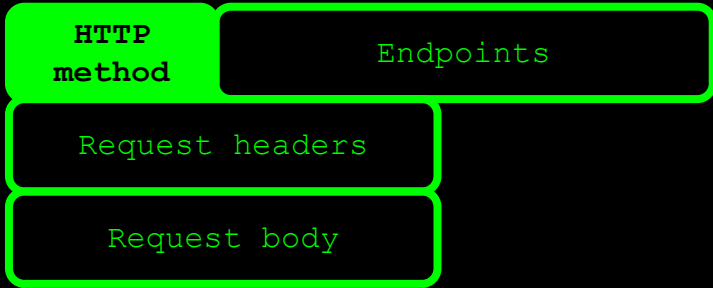
_Install Visual Studio (or any other IDE)

_Import project into your IDE

_ <https://github.com/basdijkstra/rest-assured-net-workshop>

A REST API request





HTTP methods

_GET, POST, PUT, PATCH, DELETE, OPTIONS, ...

_CRUD operations on data

POST Create

GET Read

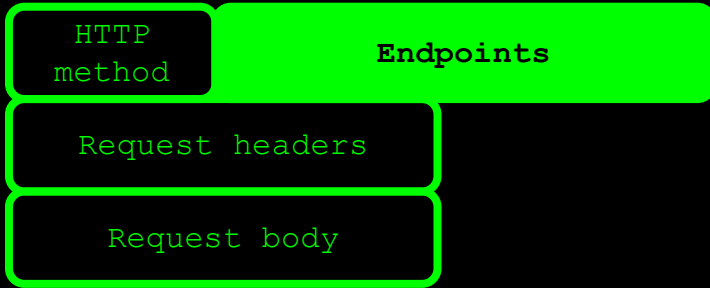
PUT / PATCH Update

DELETE Delete

...

...

_Conventions, not standards!



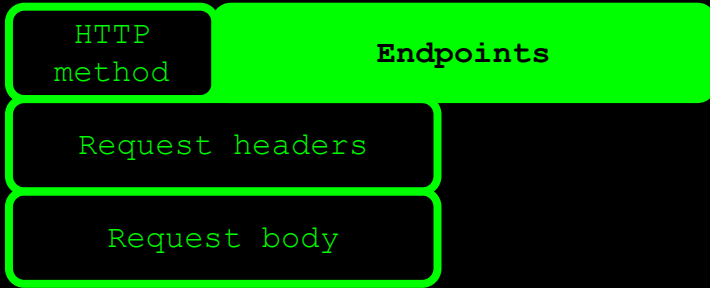
Endpoints

- Uniquely identifies the operation to perform and / or the resource to operate on

- Can contain parameters

- Query parameters

- Path parameters



Endpoint parameters

_ Path parameters

_ `http://api.zippopotam.us/us/90210`

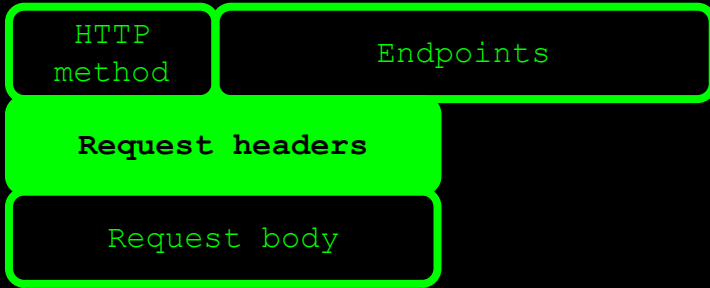
_ `http://api.zippopotam.us/ca/B2A`

_ Query parameters

_ `http://md5.jsontest.com/?text=testcaseOne`

_ `http://md5.jsontest.com/?text=testcaseTwo`

_ There is no official standard!



Request headers

- _ Key-value pairs

- _ Can contain metadata about the request body

- _ Content-Type (what data format is the request body in?)

- _ Accept (what data format would I like the response body to be in?)

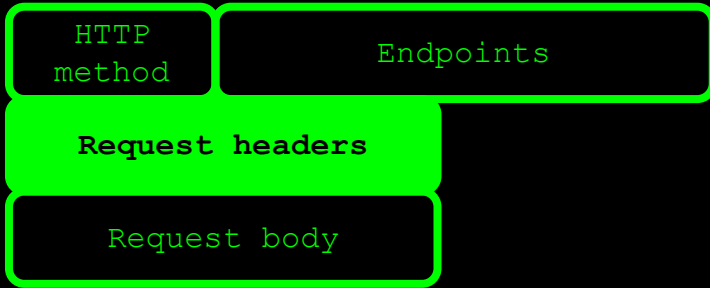
- _ ...

- _ Can contain session and authorization data

- _ Cookies

- _ Authorization tokens

- _ ...



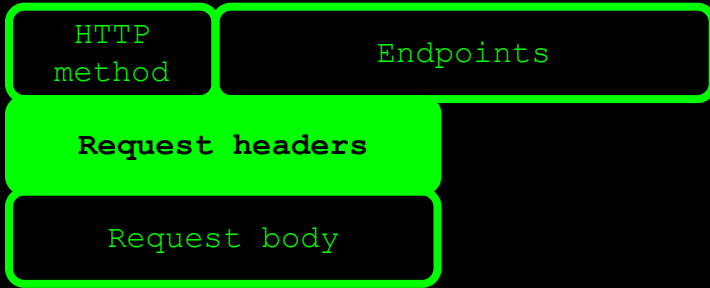
Authorization: Basic

_Username and password sent with every request

_Base64 encoded (not at all secure!)

_Ex: username = aladdin and password = opensesame

Authorization: Basic YWxhZGRpbjpvcGVuc2VzYW1l



Authorization: Bearer

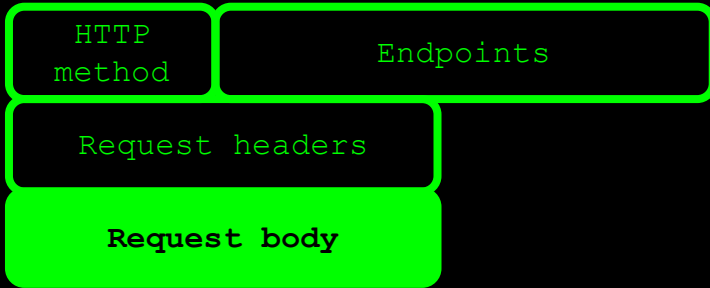
_Token with expiry date is obtained first

_Token is then sent with all subsequent requests

_Most common mechanism is OAuth(2)

_JWT is a common token format

Authorization: Bearer RST50jbzRn430zqMLgV3Ia



Request body

- Data to be sent to the provider

- REST does not prescribe a specific data format

- Most common:

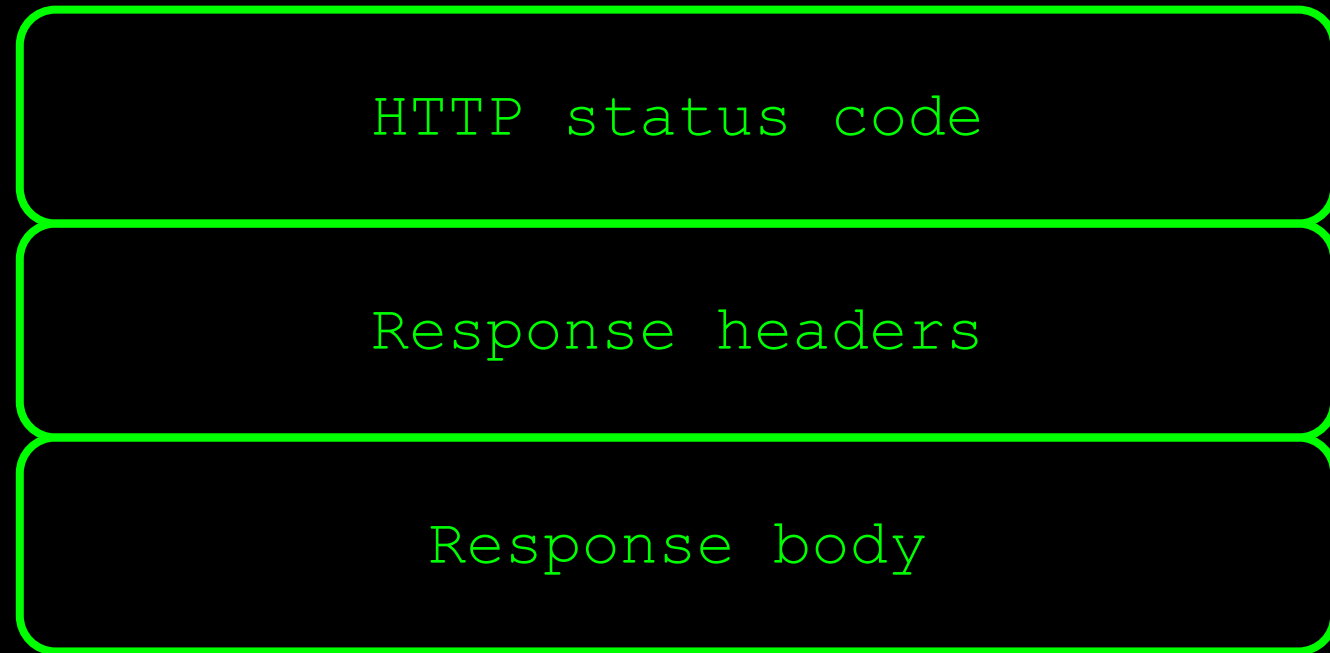
- JSON

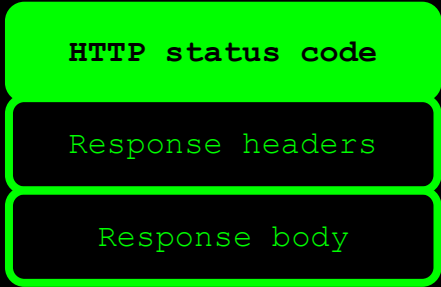
- XML

- Plain text

- Other data formats can be sent using REST, too

A REST API response





HTTP status code

— Indicates result of request processing by provider

— Five different categories

— 1XX	Informational	100 Continue
— 2XX	Success	200 OK
— 3XX	Redirection	301 Moved Permanently
— 4XX	Consumer errors	400 Bad Request
— 5XX	Provider errors	500 Internal Server Error

HTTP status code

Response headers

Response body

Response headers

- _Key-value pairs

- _Can contain metadata about the response body

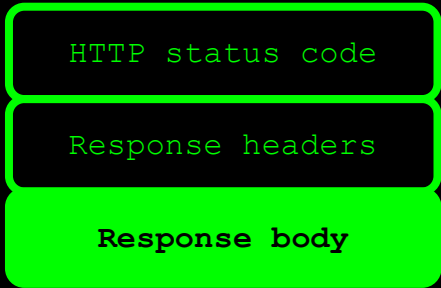
 - _Content-Type (what data format is the response body in?)

 - _Content-Length (how many bytes in the response body?)

- _Can contain provider-specific data

 - _Caching-related headers

 - _Information about the server type



Response body

- Data returned by the provider

- REST does not prescribe a specific data format

- Most common:

- JSON

- XML

- Plain text

- Other data formats can be sent using REST, too

API specification / documentation

_SOAP: WSDL (Web Services Description Language)

_REST: OpenAPI / Swagger

_REST: RAML (RESTful API Modelling Language)

_REST: WADL (Web Application Description Language)

Tools for testing RESTful APIs

_Free / open source

- _Postman

- _SoapUI

- _Code libraries (REST Assured, RestAssured.Net, RestSharp, requests, ...)

_Commercial

- _Parasoft SOAtest

- _SmartBear ReadyAPI

- _...

_Build your own (using HTTP libraries for your language of choice)

RestAssured.Net

- _C# DSL for writing tests for RESTful APIs

- _Port of REST Assured (Java)

- _Removes a lot of boilerplate code

- _Runs on top of common unit testing frameworks
 - _NUnit, xUnit, MSTest

- _Developed and maintained by Bas Dijkstra

Adding RestAssured.Net to your project

_Available as a NuGet package

NuGet Package Manager:

Browse Installed Updates **2**

restassured.net × ↻ ☐ Include prerelease ☐ Show only vulnerable

	RestAssured.Net by Bas Dijkstra C# port of the popular REST Assured library for writing tests for HTTP APIs.	4.8.0	▲
---	--	-------	---

RestAssured.Net documentation

Usage Guide

<https://github.com/basdijkstra/rest-assured-net/wiki/Usage-Guide>

Examples

<https://github.com/basdijkstra/rest-assured-net/tree/main/RestAssured.Net.Tests>

(these double as acceptance tests for the project)

A sample test

RestAssured.Net uses NUnit (this could also be xUnit or MSTest)

[Test]

0 | 0 references

```
public void GetUserData_VerifyName_ShouldBeLeanneGraham()
{
    Given()
        .When()           Make an HTTP GET call to retrieve data from the provider
        .Get("https://jsonplaceholder.typicode.com/users/1")
        .Then()
        .AssertThat()
        .Body("$.name", NHamcrest.Is.EqualTo("Leanne Graham"));
}
```

Perform an assertion on the returned response (here: on the JSON response payload)

RestAssured.Net features

- _Support for all HTTP methods (GET, POST, PUT, ...)
- _Support for Gherkin (Given/When/Then)
- _Use of NHamcrest matchers for checks (*Is.EqualTo*)
- _Use of JsonPath to select elements from a JSON response

```
[Test]
❏ | 0 references
public void GetUserData_VerifyName_ShouldBeLeanneGraham()
{
    Given()
        .When()
        .Get("https://jsonplaceholder.typicode.com/users/1")
        .Then()
        .AssertThat()
        .Body("$.name", NHamcrest.Is.EqualTo("Leanne Graham"));
}
```

About NHamcrest matchers

_Express expectations in natural language

_Examples:

`Is.EqualTo(X)`

Does the object equal X?

`Has.Item(Is.EqualTo(X))`

Does the collection contain an item equal to X?

About JsonPath

- _ JsonPath is a query language for JSON documents

- _ Similar aims and scope as XPath for XML

- _ Documentation and examples:

- _ <https://support.smartbear.com/alertsite/docs/monitors/api/endpoint/jsonpath.html>

- _ All required JsonPath expressions are given in the exercise descriptions

Checking technical response data

_ HTTP status code

_ Response Content-Type header

_ Other headers and their value

_ Cookies and their value

_ ...

_ Most methods accept
both fixed values
and NHamcrest
matchers

```
[Test]
0 | 0 references
public void GetUserData_VerifyStatusCodeAndContentType()
{
    Given()
        .When()
        .Get("https://jsonplaceholder.typicode.com/users/1")
        .Then()
        .AssertThat()
        .StatusCode(200)
        .And()
        .ContentType("application/json; charset=utf-8");
}
```

Logging request data

```
[Test]
0 references
public void LogAllRequestAndResponseData()
{
    var logConfiguration = new LogConfiguration
    {
        RequestLogLevel = RequestLogLevel.All,
        ResponseLogLevel = ResponseLogLevel.All,
    };

    Given()
        .Log(logConfiguration)
        .When()
        .Get("https://jsonplaceholder.typicode.com/users/1")
        .Then()
        .AssertThat()
        .Body("$.name", NHamcrest.Is.EqualTo("Leanne Graham"));
}
```

You can also use *RequestLogLevel.Body*, *RequestLogLevel.Headers* as well as other options

Logging request data

[Test]

0 references

```
public void LogAllRequestAndResponseData()
{
    var logConfiguration = new LogConfiguration
    {
        RequestLogLevel = RequestLogLevel.All,
        ResponseLogLevel = ResponseLogLevel.All,
    };
}
```

Given()

```
.Log(logConfiguration)
.When()
.Get("https://jsonplaceholder.typicode.com/users/1")
.Then()
.AssertThat()
.Body("$.name", NHam
```



LogAllRequestData



Source: [Examples01.cs](#) line 22



Duration: 30 ms

Standard Output:

GET <https://jsonplaceholder.typicode.com/users/1>

Content-Type: application/json; charset=utf-8

Content-Length: 0

Logging response data

[Test]

0 references

```
public void LogAllRequestAndResponseData()
{
    var logConfiguration = new LogConfiguration
    {
        RequestLogLevel = RequestLogLevel.
        ResponseLogLevel = ResponseLogLevel.
    };

    Given()
        .Log(logConfiguration)
        .When()
        .Get("https://jsonplaceholder.typicode.com/todos/1")
        .Then()
        .AssertThat()
        .Body("$.name", NHamcrest.Is.EqualTo("Mr. Brown"))
}
```

☒ LogAllResponseData

Source: [Examples01.cs](#) line 34

🕒 Duration: 41 ms

Standard Output:

HTTP 200 (OK)

```
Content-Type: application/json; charset=utf-8
```

Content-Length: 509

Date: Thu, 25 Jan 2024 11:31:41 GMT

Connection: keep-alive

```
Report-To: {"group":"heroku-nel","max_age":3600,"endpoints":[{"url":"https://heroku-nel.heroku.com"}]}
```

Reporting-Endpoints: heroku-nel=<https://nel.heroku.com/reports?>

```
Ne1: {"report_to":"heroku-ne1","max_age":3600,"success_fraction":0.5}
```

X-Powered-By: Express

```
X-Ratelimit-Limit: 1000
```

X-Ratelimit-Remaining: 999

X-Ratelimit-Reset: 1698817725

Vary: Origin, Accept-Encoding

```
Access-Control-Allow-Credentials: true
```

Cache-Control: max-age=43200

```
Pragma: no-cache
```

```
X-Content-Type-Options: nosniff
```

ETag: W/"1fd-+2Y3G3w049iSZtw5t1mzSnunngE"

Via: 1 1 veður

Our API under test

_(Simulation of) an online banking API

_Customer data (GET, POST)

_Account data (POST, GET)

_RESTful API



Demo

_How to use the test suite

_Executing your tests

_Reviewing test results

Now it's your turn!

- _Exercises > Exercises01.cs

- _Simple checks

- _Validating individual elements
 - _Validating collections and items therein
 - _Validating technical response properties

- _Stubs are predefined

- _Don't worry about the references to `http://localhost`
 - _You only need to write the tests using `RestAssured.Net`

- _Answers are in Answers > Answers01.cs

- _Examples are in Examples > Examples01.cs

Parameters in RESTful web services

_Path parameters

_ `http://api.zippopotam.us/us/90210`

_ `http://api.zippopotam.us/ca/B2A`

_Query parameters

_ `http://md5.jsontest.com/?text=testcaseOne`

_ `http://md5.jsontest.com/?text=testcaseTwo`

_There is no official standard!

Using query parameters

_GET <http://md5.jsontest.com/?text=testcase>

[Test]

0 | 0 references

public void UseQueryParameter()

{

Define a query parameter and its value

Given()

.QueryParam("text", "testcase")

.When()

.Get("http://md5.jsontest.com")

.Then()

.Body("\$.md5", NHamcrest.Is.EqualTo("7489a25fc99976f06fecb807991c61cf"));

}

Using path parameters

_GET http://jsonplaceholder.typicode.com/users/1

[Test]

0 | 0 references

```
public void UsePathParameter()
```

```
{  
    // Define a (custom) path parameter name  
    // and the parameter value
```

```
    Given()
```

```
        .PathParam("userId", 1)
```

Define the location of the path parameter
using the chosen name between []

```
        .When()
```

```
        .Get("https://jsonplaceholder.typicode.com/users/[userId]")
```

```
        .Then()
```

```
        .Body("$.name", NHamcrest.Is.EqualTo("Leanne Graham"));
```

```
}
```

Exchange data between consumer and provider

GET to retrieve data from provider, POST to send data to provider, ...

APIs are all about data

Business logic and calculations often exposed through APIs

Run the same test more than once...

... for different combinations of input and
expected output values

Parameterized testing

More efficient to do this at the API level...

... as compared to doing this at the UI level

'Feeding' test data to your test

Define test cases using the [TestCase] attribute (one for every iteration with test data values separated by commas)

```
[TestCase(1, "Leanne Graham", TestName = "User with ID 1 is Leanne Graham")]  
[TestCase(2, "Ervin Howell", TestName = "User with ID 2 is Ervin Howell")]  
[TestCase(3, "Clementine Bauch", TestName = "User with ID 3 is Clementine Bauch")]
```

0 references

```
public void CheckNameForUser(int userId, string expectedUserName)  
{  
    Given()  
        .PathParam("userId", userId)  
        .When()  
        .Get("https://jsonplaceholder.typicode.com/users/[userId]")  
        .Then()  
        .Body("$.name", NHamcrest.Is.EqualTo(expectedUserName));  
}
```

Use parameters to pass the test data values into the method

Use parameters in the test method where appropriate

Running the parameterized test

```
[TestCase(1, "Leanne Graham", TestName = "User with ID 1 is Leanne Graham")]  
[TestCase(2, "Ervin Howell", TestName = "User with ID 2 is Ervin Howell")]  
[TestCase(3, "Clementine Bauch", TestName = "User with ID 3 is Clementine Bauch")]
```

0 references

```
public void CheckNameForUser(int userId, string expectedUserName)  
{  
    Given()  
        .PathParam("userId", userId)  
        .When()  
        .Get("https://jsonplaceholder.typicode.com/users/[userId]")  
        .Then()  
        .AssertThat(response.StatusCode == HttpStatusCode.OK, "User not found");  
}
```

▲ ✓ Examples02 (3)	1,6 sec
✓ User with ID 1 is Leanne Graham	966 ms
✓ User with ID 2 is Ervin Howell	342 ms
✓ User with ID 3 is Clementine Bauch	337 ms

The test method is run three times, once for each iteration (or 'test case')

Now it's your turn!

- _Exercises > Exercises02.cs

- _Parameterized tests

- _Creating iterations using the [TestCase] annotation

- _Using parameterized data to call the right URI

- _Using parameterized data in assertions

- _Answers are in Answers > Answers02.cs

- _Examples are in Examples > Examples02.cs

Authentication

- _Securing APIs

- _Most common authentication schemes:

 - _Basic authentication (username / password)

 - _Token-based, often using OAuth(2)

Basic authentication

[Test]

0 | 0 references

```
public void UseBasicAuthentication()
```

```
{
```

This will add the Authorization header to the request, with the appropriate value

```
    Given()
```

```
        .BasicAuth("username", "password")
```

```
        .When()
```

```
        .Get("https://my.secure/api")
```

```
        .Then()
```

```
        .StatusCode(200);
```

```
}
```

OAuth (2)

[Test]

0 | 0 references

```
public void UseOAuth2Authentication()
```

```
{
```

```
    Given()
```

The authentication token is typically retrieved prior to running the tests to ensure that a valid token is used

```
        .OAuth2("my_authentication_token")
```

```
        .When()
```

```
        .Get("https://my.very.secure/api")
```

```
        .Then()
```

```
        .StatusCode(200);
```

```
}
```

Sharing variables between tests

_Example: tokens, uniquely generated IDs

_First call returns a unique value (e.g. a new user ID)

_Second call needs to use this generated value

_Since there's no way to predict the value, we need to capture and reuse it

Sharing variables between tests

[Test]

0 | 0 references

```
public void CaptureAndReuseUniqueId()
```

```
{    The return value can be stored in a variable...
```

```
    string userId = (string)Given()
```

```
        .When()
```

```
        .Post("https://my.user.api/user")
```

```
        .Then()
```

```
        .Extract()
```

```
        .Body("$.id");
```

Body() takes a JsonPath expression to extract the required value

```
    Given()
```

```
        .PathParam("userId", userId)
```

```
        .When()    ... and reused at a later point in time
```

```
        .Get("https://my.user.api/user/[userId]")
```

```
        .Then()
```

```
        .StatusCode(200);
```

```
}
```

RequestSpecifications

`_Reuse` shared properties shared by many calls

`_Base URI`

`_Port`

`_Headers, authentication, cookies`

`_...`

Defining and using a RequestSpecification

```
private RequestSpecification? requestSpec;
```

[SetUp]

0 references

```
public void CreateRequestSpecification()
```

```
{
```

```
    requestSpec = new RequestSpecBuilder()
```

```
        .WithBaseUrl("https://jsonplaceholder.typicode.com")
```

```
        .WithContentType("application/json")
```

```
        .WithOAuth2("my_authentication_token")
```

```
        .Build();
```

```
}
```

Build your RequestSpecification using a fluent Builder pattern...

[Test]

0 references

```
public void UseRequestSpecification()
```

```
{
```

```
    Given()
```

```
        .Spec(requestSpec)
```

```
        .When()
```

```
        .Get("/users/1")
```

```
        .Then()
```

```
        .StatusCode(200);
```

```
}
```

... and use it by calling `Spec()` in the `Given()` section of your test

Now it's your turn!

- _Exercises > Exercises03.cs

- _Reuse shared values

- _Apply value reuse as shown in the slides

- _Use basic and OAuth authentication schemes

- _Extract common values to a RequestSpecification

- _Answers are in Answers > Answers03.cs

- _Examples are in Examples > Examples03.cs

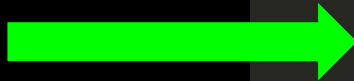
Working with XML responses

_REST APIs can return XML responses, too

_RestAssured.Net can work with this data format, too

_Use XPath instead of JsonPath to select response body element(s)

If we want to verify that the value of the 'type' field is equal to CHECKING ...



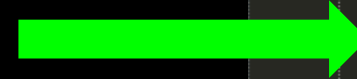
```
<account>
  <id>12345</id>
  <customerId>12212</customerId>
  <type>CHECKING</type>
  <balance>-2300.00</balance>
</account>
```

[Test]

0 references

```
public void GetAccount12345_CheckType_ShouldBeChecking()
{
    Given()
        .When()
        .Get("https://parabank.parasoft.com/parabank/services/bank/accounts/12345")
        .Then()
        .StatusCode(200) ... we can get the element value with this XPath expression
        .Body("//type", Namcrest.Is.EqualTo("CHECKING"));
}
```

If we want to verify that the list of all CHECKING account ids contains the value 12678 ...



```
<accounts>
  <account>
    <id>12567</id>
    <customerId>12212</customerId>
    <type>CHECKING</type>
    <balance>100.00</balance>
  </account>
  <account>
    <id>12678</id>
    <customerId>12212</customerId>
    <type>SAVINGS</type>
    <balance>-100.00</balance>
  </account>
  <account>
    <id>12789</id>
```

[Test]

0 references

```
public void GetAccountsForCustomer12212_CheckSavingsAccounts_ShouldContain12678()
{
    Given()
        .When()
        .Get("https://parabank.parasoft.com/parabank/services/bank/customers/12212/accounts")
        .Then()
        .StatusCode(200)
        .Body(
            "//account/type[text()='SAVINGS']/parent::account/id",
            NHamcrest.Has.Item(NHamcrest.Is.EqualTo(12678))
        );
}
```

... we can do that like this

Now it's your turn!

_Exercises > Exercises04.cs

_Work with APIs returning XML response bodies

_Verify XML response body element values

_Create the required XPath expressions yourself

_Answers are in Answers > Answers04.cs

_Examples are in Examples > Examples04.cs

(De-)serialization of objects

- _ RestAssured.Net is able to convert objects directly to XML or JSON (and back)
- _ Useful when dealing with API payloads
 - _ Creating request body payloads
 - _ Processing response body payloads
- _ No need for additional configuration or libraries

Example: serialization

_Class / DTO / POCO / ... representing a blog post

```
internal class Post
```

```
{
```

```
    [JsonProperty("userId")]
```

```
    1 reference | 0/1 passing
```

```
    public int UserId { get; set; }
```

```
    [JsonProperty("title")]
```

```
    3 references | 0/3 passing
```

```
    public string Title { get; set; } = string.Empty;
```

```
    [JsonProperty("body")]
```

```
    1 reference | 0/1 passing
```

```
    public string Body { get; set; } = string.Empty;
```

```
}
```

RestAssured.Net uses Json.NET for (de-)serialization, which means that all Json.NET attributes can be used

[JsonProperty] defines the name of the property as it appears in JSON

Example: serialization

[Test]

0 references

```
public void SerializePostObjectToJson()
```

```
{
```

```
    Post post = new Post
```

Create a new Post object ...

```
{
```

```
    UserId = 1,
```

```
    Title = "My new blog post",
```

```
    Body = "This is an awesome piece of content"
```

```
},
```

```
    Given()
```

... then pass it as a request body using *Body()*...

```
        .Body(post)
```

```
        .When()
```

```
        .Post("https://jsonplaceholder
```

```
        .Then()
```

```
        .StatusCode(201);
```

```
}
```

```
{
```

```
  "userId": 1,
```

```
  "title": "My new blog post",
```

```
  "body": "This is an awesome piece of content"
```

```
}
```

... and RestAssured.Net will serialize it to JSON automatically

Serializing anonymous objects

[Test]

0 references

```
public void SerializeAnonymousObjectToJson()
```

```
{
```

```
    var post = new
```

Create a new anonymous object ...

```
    {
```

```
        userId = 1,
```

```
        title = "My new blog post",
```

```
        body = "This is an awesome piece of content"
```

This is useful for one-off request payloads, as it doesn't require you to create a separate class to serialize from

```
    };
```

```
    Given()
```

... then pass it as a request body using *Body()*...

```
        .Body(post)
```

```
        .When()
```

```
        .Post("https://jsonplaceholder
```

```
        .Then()
```

```
        .StatusCode(201);
```

```
{
```

```
    "userId": 1,
```

```
    "title": "My new blog post",
```

```
    "body": "This is an awesome piece of content"
```

```
}
```

... and RestAssured.Net will serialize it to JSON automatically

Example: deserialization

[Test]

0 references

```
public void DeserializeJsonToPostAfterVerification()
```

```
{    ... store the deserialized response payload in an object of that type...
```

```
    var post = Given()
```

```
        .When()
```

```
        .Get("https://jsonplaceholder.typicode.com/posts/1")
```

```
        .Then()    Perform response verifications as usual...
```

```
        .StatusCode(200)
```

```
        .DeserializeTo<Post>();    Specify the type to deserialize  
                                   to using DeserializeTo<T>()...
```

```
    Assert.That(post!.Title, Contains.Substring("sunt aut facere"));
```

```
}
```

... and then use it in the remainder of your test method as required

Example: deserialization (without initial checks)

[Test]

0 references

```
public void DeserializeJsonToPost()
{
    ... store the deserialized response payload in an object of that type...
    var post = Given()
        .When()
        .Get("https://jsonplaceholder.typicode.com/posts/1")
        .DeserializeTo<Post>();
    Assert.That(post!.Title, Contains.Substring("sunt aut facere"));
}
... and then use it in the remainder of your test method as required
```

Specify the object type to deserialize to using *DeserializeTo<T>()*...

Now it's your turn!

_Exercises > Exercises05.cs

_Practice with (de-)serialization

_You don't need to create or adapt the classes / DTOs yourself, that has been done for you already. By all means go ahead and inspect them, though

_Answers are in Answers > Answers05.cs

_Examples are in Examples > Examples05.cs

One challenge with
'traditional' REST APIs

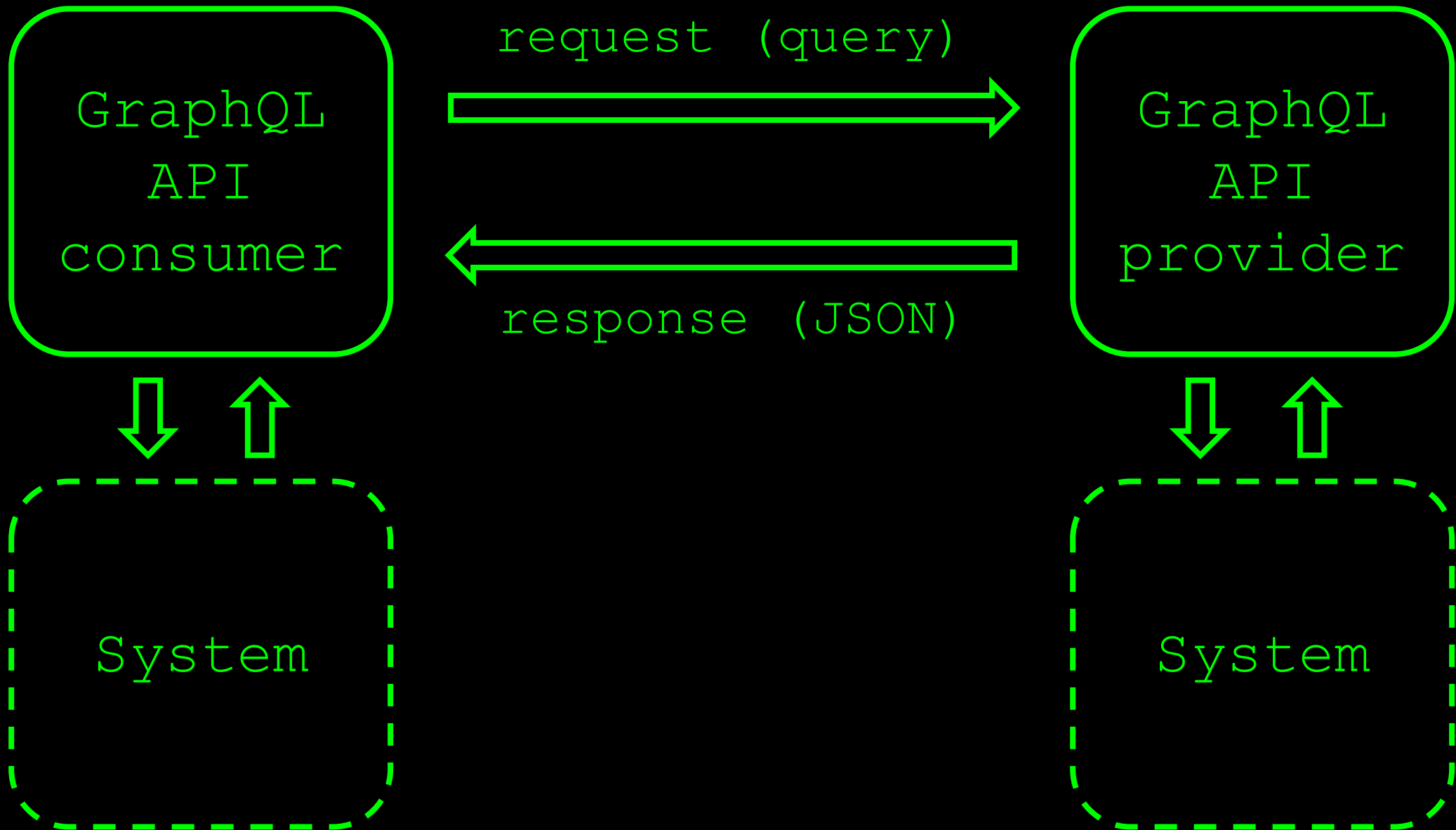
Query language for APIs...

... as well as a runtime to fulfill them

GraphQL

"Ask for what you need,
and get exactly that"

<https://graphql.org>



Create a valid GraphQL query...

... and send it in the request body (*query*)

Sending a GraphQL query

“Ask for what you need, and get exactly that”

Request payload is still in JSON format

These are 'regular' REST responses, with...

- ... an HTTP status code, ...

GraphQL API responses

- ... response headers...

- ... and a JSON response body
containing the requested data

```
private readonly string hardcodedGraphQLQuery =
```

```
@"query GetCountryData {  
  country(code: ""NL"") {
```

```
    name  
    capital  
    currency
```

```
  }  
}"
```

```
[Test]
```

```
0 references
```

```
public void UseHardCodedValuesInQuery CheckTheCapital()
```

```
{
```

```
    GraphQLRequest request = new GraphQLRequestBuilder()  
        .WithQuery(this.hardcodedGraphQLQuery)  
        .WithOperationName("GetCountryData")  
        .Build();
```

```
    Given()
```

```
        .GraphQL(request)
```

```
        .When()
```

```
        .Post("https://countries.trevorblades.com/graphql")
```

```
        .Then()
```

```
        .StatusCode(200)
```

```
        .Body("$.data.country.capital", Hamcrest.Is.EqualTo("Amsterdam"));
```

```
}
```

Build our GraphQL request payload ...

... add it to the API request ...

... and verify the JSON response body


```
private readonly string parameterizedGraphQLQuery =
```

```
@"query GetCountryData($country: ID!) {
```

```
  country(code: $
```

```
    name
```

```
    capital
```

```
    currency
```

```
    languages {
```

```
      code
```

```
      name
```

```
    }
```

```
  }
```

```
}"
```

```
[TestCase("NL", "Amsterdam", TestName = "The capital of NL is Amsterdam")]
```

```
[TestCase("IT", "Rome", TestName = "The capital of IT is Rome")]
```

```
[TestCase("CA", "Ottawa", TestName = "The capital of CA is Ottawa")]
```

```
0 references
```

```
public void UseParametersInQuery_CheckTheCapital  
(string countryCode, string expectedCapital)
```

```
Dictionary<string, object> variables = new Dictionary<string, object>
```

```
{
```

```
    { "country", countryCode }, Set the query variables ...
```

```
};
```

```
GraphQLRequest request = new GraphQLRequestBuilder()
```

```
    .WithQuery(this.parameterizedGraphQLQuery)
```

```
    .WithOperationName("GetCountryData")
```

```
    .WithVariables(variables)
```

```
    .Build();
```

... build our GraphQL request payload ...

```
Given()
```

```
    .GraphQL(request)
```

... and add it to the API request

```
    .When()
```

```
    .Post("https://countries.trevorblades.com/graphql")
```

```
    .Then()
```

```
    .StatusCode(200)
```

```
    .Body("$.data.country.capital", NHamcrest.Is.EqualTo(expectedCapital));
```

```
}
```

Now it's your turn!

- _Exercises > Exercises06.cs

- _Work with GraphQL APIs

- _Invoke an endpoint with a non-parameterized query

- _Invoke an endpoint with a parameterized query

- _Answers are in Answers > Answers06.cs

- _Examples are in Examples > Examples06.cs

Adding abstraction layers

- _ Once your test suite grows, you'll find yourself reusing certain requests over and over
- _ Writing them in full every time you need them means code duplication and decreased maintainability
- _ Solution: add an abstraction layer on top of (parts of) the RestAssured.Net code

This class should not be instantiated in tests

Step 1: Create a ClientBase

```
public abstract class ClientBase
```

```
{  
    private string baseUri;
```

1 reference

```
protected ClientBase(string baseUri)
```

Parameters that might change for different environments go here

```
{
```

```
    this.baseUri = baseUri;
```

If these are always the same they can also be hardcoded

```
}
```

2 references

```
public RequestSpecification GetRequestSpec()
```

```
{
```

```
    return new RequestSpecBuilder()
```

```
        .WithBaseUri(baseUri)
```

```
        .WithContentType("application/json")
```

```
        .Build();
```

```
}
```

```
}
```

The RequestSpecification contains properties shared among all requests

```

public class PostClient : ClientBase
{
    Inherit shared properties from the base class
    private static readonly string BaseUri = "https://jsonplaceholder.typicode.com";

    1 reference
    public PostClient() : base(BaseUri)
    {
        Initialize the class and the base class
    }

    1 reference | 0/1 passing
    public VerifiableResponse GetPost(int postId)
    {
        return Given()
            .Spec(base.GetRequestSpec())
            .When()
            .Get($"/posts/{postId}");
    }

    1 reference | 0/1 passing
    public HttpResponseMessage CreatePost(Post post)
    {
        return Given()
            .Spec(base.GetRequestSpec())
            .Body(post)
            .When()
            .Post("/posts")
            .Then()
            .StatusCode(201)
            .Extract().Response();
    }
}

```

Step 2:
Create a
Client class

Create a method that performs an
action that is repeated across tests

Here, it's retrieving a post based on
the post ID

It returns a VerifiableResponse (this
is a RestAssured.Net class)

Alternatively, you can create methods that
return the 'raw' HttpResponseMessage (this
is a System.Net.Http class)

This way, you can also perform basic checks
before returning the response object, if
you want

```
public class Examples05 : TestBase
```

```
{
```

```
private readonly PostClient postClient = new PostClient();
```

```
[Test]
```

Create a new client instance

```
0 | 0 references
```

```
public void ApplyClientTestModel_ReturnVerifiableResponse_CheckStatusCodeAndResponseHeader()
```

```
{
```

```
postClient.GetPost(1)
```

```
.Then()
```

```
.StatusCode(200)
```

```
.ContentType(NHamcrest.Contains.String("application/json"));
```

```
}
```

Call client methods to perform repeatable actions

```
[Test]
```

```
0 | 0 references
```

```
public void ApplyClientTestModel_ReturnHttpResponseMessage_CheckStatusCodeAndResponseHeader()
```

```
{
```

```
Post post = new Post
```

```
{
```

```
    UserId = 1,
```

```
    Title = "My new blog post",
```

```
    Body = "This is the body of my brand new blog post."
```

```
};
```

```
var response = postClient.CreatePost(post);
```

```
Assert.That(response.StatusCode, Is.EqualTo(HttpStatusCode.Created));
```

```
response.Content.Headers.TryGetValues("Content-Type", out IEnumerable<string>? values);
```

```
Assert.That(values!.First(), Does.Contain("application/json"));
```

Step 3: Use the Client class in your tests

Returning a VerifiableResponse enables using RestAssured.Net verifications and a fluent syntax

Working with an HttpResponseMessage requires a bit more work to perform verifications on the response

You do have access to the raw response, though, which could be beneficial in certain cases

Now it's your turn!

_Exercises > Exercises07.cs

_Practice with adding abstraction layers to your tests

_The ClientBase has been created for you

_First, define the appropriate methods in the AccountClient

_Then, complete the tests using the AccountClient methods

_Answers are in Answers > Answers07.cs

_Examples are in Examples > Examples07.cs



Contact

Email: bas@ontestautomation.com

Website: <https://www.ontestautomation.com/training>

LinkedIn: <https://www.linkedin.com/in/basdijkstra>